Questions::

1. Why do we use `IActionResult` instead of `ActionResult`?

- **Answer:** `IActionResult` is an interface that provides maximum flexibility for action methods. It acts as a contract that allows an action to return various types of HTTP responses (such as `View()`, `Json()`, `NotFound()`, or `BadRequest()`) depending on the execution outcome.
- **Scenario / Problem:** Consider an action method that retrieves a student by ID. If the student exists, the method returns `Ok(student)` (200 OK with JSON). If the student is not found, it returns `NotFound()` (404 Not Found). Using `IActionResult` allows returning both distinct response types from the same method without type-mismatch compile errors.

2. What do the `HttpContext` Request and Response messages consist of?

- **Request Message Components:**
    1. **Request Line:** Contains the HTTP Method (e.g., `GET`, `POST`), the Request URL/Path, and the HTTP version.
    2. **Headers:** Key-value pairs containing metadata about the request and client (e.g., `User-Agent`, `Authorization`, `Accept`).
    3. **Body:** The payload data sent by the client, commonly present in `POST` or `PUT` requests (formatted as JSON, XML, or Form Data).
- **Response Message Components:**

    1. **Status Line:** Contains the HTTP Status Code and phrase (e.g., `200 OK`, `404 Not Found`, `500 Internal Server Error`).
    2. **Headers:** Metadata provided by the server (e.g., `Content-Type`, `Set-Cookie`, `Server`).
    3. **Body:** The actual content returned to the client (e.g., HTML code, JSON data, or image files).

3. What is the difference between HTTP and HTTPS?

- **HTTP (Hypertext Transfer Protocol):** Transmits data as unencrypted plain text over the default port **`80`**. It is insecure and vulnerable to interception or eavesdropping.
- **HTTPS (HTTP Secure):** Encrypts all transmitted data using **SSL/TLS certificates** over the default port **`443`**. It ensures data confidentiality, privacy, and integrity.

4. What are Segments and Fragments in a URL? (With a real URL example)

- **Real URL Example:**
  `[https://example.com/products/electronics#specifications](https://example.com/products/electronics#specifications)`

- **Segments:** The path components separated by slashes (`/`) that define the hierarchical route on the server (e.g., `products` and `electronics` are path segments mapping to controllers and actions).
- **Fragments:** The portion following the hash symbol (`#`) (e.g., `#specifications`). It identifies a specific internal location or section within the loaded document and is processed entirely on the client side without being sent to the server.

5. What are Builder and Dependency Injection? (With a real-life example)

- **Builder Pattern:** A creational design pattern used to construct complex objects step by step (e.g., `WebApplication.CreateBuilder(args)` in ASP.NET Core used to configure server services and middleware).
- **Dependency Injection (DI):** A design pattern that implements Inversion of Control (IoC), where object dependencies are provided (injected) from the outside rather than being created internally using `new`.
- **Real-Life Example:** Think of ordering food at a **restaurant**. Instead of going into the kitchen to prepare and assemble the meal yourself (manually creating dependencies), you request the meal from the **waiter (DI Container)**, who delivers the prepared food directly to your table to consume.

6. What is the difference between Web Pages (Razor Pages) and MVC?

- **Razor Pages:**
  - **Architecture:** It is page-focused. It pairs the user interface (`.cshtml`) directly with its corresponding code-behind file (`.cshtml.cs`).
  - **Best Used For:** Simple, page-centric, or form-focused workflows where each page operates relatively independently.
- **ASP.NET Core MVC:**
  - **Architecture:** It follows a strict Separation of Concerns pattern by dividing the application into Models, Views, and Controllers.
  - **Best Used For:** Large-scale, complex applications requiring high modularity, flexible routing, and extensive unit testing capabilities.
- **Two Business Cases:**

  1. **Razor Pages Case:** Building a simple company landing page, a basic feedback/contact form, or a small admin entry form where each page handles its own logic directly.
  2. **MVC Case:** Building an enterprise-grade E-Commerce platform or healthcare management portal with multi-tier role management, complex business logic, and multiple development teams working on separate components.

7. What is `Content-Type` in a response message, where is it used, and why?

- **Definition:** `Content-Type` is an HTTP response header that specifies the media type (MIME type) of the payload data returned in the response body (e.g., `application/json`, `text/html`).
- **Where & Why It Is Used:** It is included in server responses to inform the client (browser or consumer application) how to correctly parse, render, and process the response body.

8. Performance Optimization Concepts

1. **Minification:** The process of removing unnecessary characters (whitespace, comments, newlines) and shortening variable names in JS and CSS files to reduce file sizes for faster network transmission.
2. **Web Bundle:** Combining multiple client-side files (e.g., multiple CSS or JS files) into a single file to reduce the total number of HTTP requests made by the browser.
3. **Webpack:** A static module bundler for JavaScript applications that processes, bundles, and optimizes assets (JS, CSS, images) for production delivery.
4. **Lazy Loading (Client-Side):** A technique that delays the loading of non-critical resources (e.g., images or modules) until they are actually needed on the screen, improving initial page load performance.